Report

Building a Small GPT From Scratch: A Transformer Deep Dive



Department of AI

AI Engineering Assignment

Group Members:

Dania Khan 23k-0072

Emaan Arshad 23i-2560

Zunaira Amjad 23k-0013

Code: *https://github.com/daniakhan123/Mini-gpt*

Live demo: *https://mini-gpt-yfoz.onrender.com/docs*

1. Problem and Scenario

A small applied AI team wants a text-generation service for a specialised organisation. The organisation's private corpus is too small to train a large foundation model, but it may be enough to teach a compact model the style and vocabulary of a narrow domain. The practical question is: can a small Transformer, trained from scratch, learn the next-token structure of a domain corpus well enough to generate coherent text?

To answer it we built MiniGPT, a miniature GPT-style causal language model. Every Transformer component (attention, multi-head split/merge, positional encoding, residual blocks, normalisation, feed-forward network, LM head, loss, generation) is our own PyTorch code. No pretrained checkpoint and no high-level model class from the transformers library is used. PyTorch is used only for tensors, autograd, linear layers, LayerNorm, softmax and the optimiser, which the assignment allows.

The pipeline is: raw text, tokenizer, token IDs and batches, token embedding plus position information, causal Transformer blocks, vocabulary logits, next-token prediction, generated text.

2. Transformer Foundations

2.1 Why Transformers: sequential vs parallel depth

A recurrent network processes a sequence of length T one step at a time, because the hidden state at step t depends on step t-1. Its **sequential depth is O(T)**, so the steps cannot be run in parallel during training, and information from distant tokens must survive many state updates. Self-attention connects every position to every other position in a single layer. Its **sequential depth is O(1)** per layer, so all positions are computed in parallel, and any two tokens are one step apart.

The price is memory and computation: the attention score matrix is T x T per head, so memory grows as **$O(T^2)$**. Our small measurement (Section 6.5) shows this trade-off directly.

2.2 Scaled dot-product self-attention

Each token vector x is projected into a query Q, key K, and value V. With d_k the key dimension:

scores = (Q K^T) / sqrt(d_k)
weights = softmax(scores) (row-wise)
output = weights V

Dividing by sqrt(d_k) keeps the dot products from growing with dimension, which would push the softmax into saturated regions with tiny gradients. For causal language modelling, we add a mask before the softmax: scores at positions j > i are set to -inf, so their softmax weight is exactly 0.

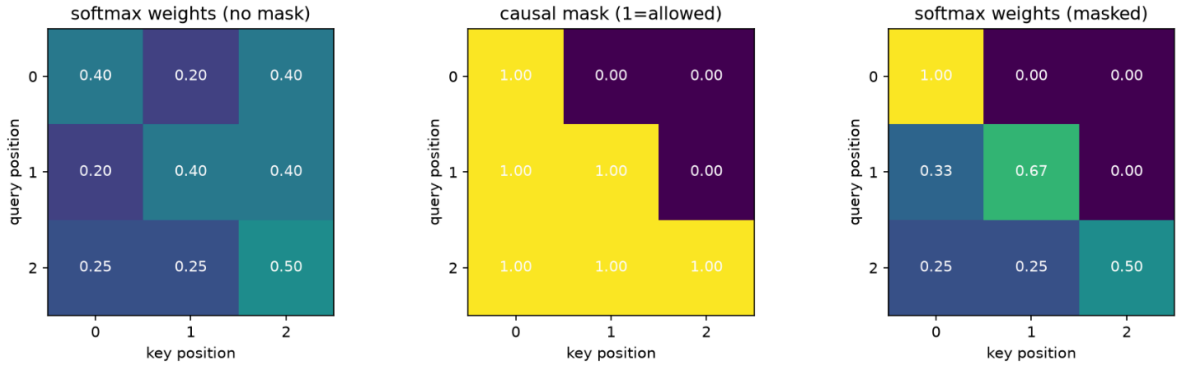
2.3 Worked hand calculation (3 tokens, d_k = d_v = 2)

We chose Q = K = [[1,0],[0,1],[1,1]] and V = [[1,0],[0,2],[1,1]].

Step	Result (rows = tokens 1, 2, 3)
Q K^T	[[1,0,1],[0,1,1],[1,1,2]]
Divide by sqrt(2)	[[0.7071,0,0.7071],[0,0.7071,0.7071],[0.7071,0.7071,1.4142]]
Row-wise softmax	[[0.4011,0.1978,0.4011],[0.1978,0.4011,0.4011],[0.2483,0.2483,0.5035]] (each row sums to 1)
Weights x V (no mask)	[[0.8022,0.7967],[0.5989,1.2033],[0.7517,1.0000]]
Causal-masked scores	[[0.7071,-inf,-inf],[0,0.7071,-inf],[0.7071,0.7071,1.4142]]
Masked weights	[[1,0,0],[0.3302,0.6698,0],[0.2483,0.2483,0.5035]]
Masked output	[[1.0000,0.0000],[0.3302,1.3395],[0.7517,1.0000]]

Sample check: for token 1, exp(0.7071) = 2.028 and exp(0) = 1, so the unmasked weights are 2.028/5.056 = 0.4011, 1/5.056 = 0.1978 and 0.4011.

What the mask changes. Without the mask, every token mixes in information from future tokens. With it, token 1 sees only itself (its output equals V[0]), token 2 averages V[0] and V[1], and token 3 sees all three. Token 3 is unchanged because it was always allowed to see everything before it.



2.4 Multi-head attention

With d_model = 128 and H = 4 heads, each head has d_head = 32. The input x of shape (B, T, 128) is projected to Q, K, V of the same shape, split into heads (B, 4, T, 32), attended per head to give weights (B, 4, T, T) and outputs (B, 4, T, 32), concatenated back to (B, T, 128), and mixed by an output projection W_o. Our printed trace for B=2, T=10:

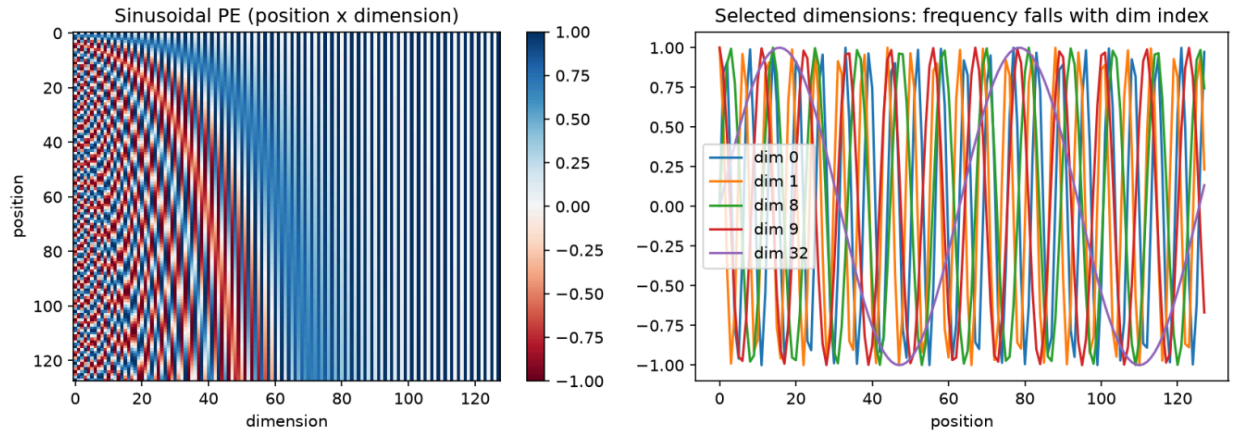
Stage	Shape
input x	(2, 10, 128)
Q/K/V projection	(2, 10, 128)
split heads Q	(2, 4, 10, 32)
attention weights	(2, 4, 10, 10)
per-head output	(2, 4, 10, 32)
concat heads	(2, 10, 128)
output projection W_o	(2, 10, 128)

Heads let the model attend to different kinds of relationships in parallel at no extra parameter cost, because the 128 dimensions are partitioned, not multiplied.

```
=== Figure 3: multi-head tensor-shape trace ===
  input x                 (2, 10, 128)
  Q/K/V projection        (2, 10, 128)
  split heads Q           (2, 4, 10, 32)
  attention weights       (2, 4, 10, 10)
  per-head output         (2, 4, 10, 32)
  concat heads            (2, 10, 128)
  output projection W_o   (2, 10, 128)
  (B=2, T=10, d_model=128, H=4, d_head=32)
```

2.5 Positional encoding

Self-attention is permutation-invariant, so order must be injected. We implement the sinusoidal encoding: PE(pos, 2i) = sin(pos / 10000^(2i/d_model)) and PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model)). It is added to the token embeddings. Low dimensions oscillate quickly and high dimensions slowly, so each position gets a unique, smoothly varying signature, and relative offsets correspond to simple linear transformations. It has no parameters (the model summary shows pos: 0). A learned-position variant is also available in the code for the optional ablation.



2.6 The Transformer block

Each decoder block applies, with residual connections around both sub-layers:

x = x + MultiHeadAttention(LayerNorm(x)) (causal)
x = x + FFN(LayerNorm(x)), where FFN(x) = W2 GELU(W1 x)

Residual paths let gradients flow directly through depth, LayerNorm stabilises activations, and the FFN (hidden size 4 x d_model = 512) adds per-token non-linear capacity. The model ends with a final LayerNorm and an LM head projecting d_model to vocabulary logits.

2.7 BERT (MLM) vs GPT (causal LM)

BERT-style masked language modelling hides random tokens and predicts them using context from **both sides**, so its attention mask is all ones (bidirectional). GPT-style causal modelling predicts the next token from the **left context only**, so its mask is lower-triangular. If a next-token model were allowed bidirectional attention, the target token at position t+1 would already be visible in the input at position t+1, and the model could copy it instead of learning to predict. Section 6.6 demonstrates this leak empirically.

3. Implementation

Component	File	Notes
Tokenizer	src/tokenizer.py	Deterministic character-level tokenizer
Dataset/batches	src/dataset.py	Contiguous fixed-length windows, shifted targets
Attention + multi-head	src/attention.py	Explicit split/merge, optional causal mask
Positional encoding	src/positional_encoding.py	Sinusoidal (learned optional)
Block and model	src/transformer_block.py, src/gpt.py	Pre-norm block, LM head, optional weight tying
Training	src/train.py	AdamW, warm-up plus cosine schedule, checkpointing, overfit mode
Generation	src/generate.py	Temperature, top-k, top-p
API	src/api.py	FastAPI: /health, /info, /generate
Tests	tests/	Attention, mask, shapes, overfit

Tokenizer. Character-level, with a vocabulary of 67 symbols: the 65 distinct characters of the corpus plus 2 special tokens. A character-level vocabulary keeps the output layer tiny and removes out-of-vocabulary problems, at the cost of long sequences and a model that must learn spelling.

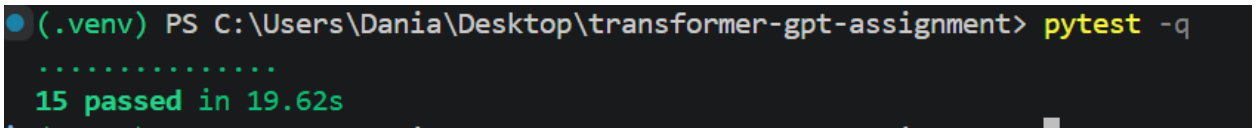
Loss and target shifting. From a window of context_length + 1 tokens, the input is tokens[0:T] and the target is tokens[1:T+1], so the target at position t is the token following the input at position t. The loss is mean token-level cross-entropy over the vocabulary logits. Perplexity is exp(loss).

Model summary (Figure 5). vocab 67, context 128, d_model 128, 4 heads (d_head 32), 4 layers, FFN 512, dropout 0.1, sinusoidal positions, weights not tied.

Module	Parameters
Token embedding	8,576
Positional encoding	0
4 Transformer blocks (198,272 each)	793,088
Final LayerNorm	256
LM head	8,576
Total	**810,496 (0.81 M)**

Per block: attention 4 x (128x128 + 128) = 66,048, FFN 131,712, two LayerNorms 512. Note: the demo model printed by experiments/architecture_evidence.py uses a default vocabulary of 65 and so reports 809,984 parameters. The trained model has vocabulary 67 and 810,496 parameters; the latter is the figure used throughout this report.

Tests. The suite has 15 tests covering attention shapes, agreement with PyTorch's reference attention, a case where future positions are forbidden from receiving attention, multi-head shape checks, and a tiny-overfit test.



4. Dataset and Preprocessing

We use **Tiny Shakespeare** (about 1.1 MB of text from Shakespeare's plays), downloaded automatically by the training script. After tokenisation, there are 1,003,854 training tokens and 111,540 validation tokens (a 90/10 contiguous split). Context length is 128 characters. No filtering was applied. The corpus is public, so no private data is involved.

5. Training Setup

Item	Value
Optimiser	AdamW, peak learning rate 3e-4, weight decay 0.1, gradient clipping 1.0
Schedule	200 warm-up steps, then cosine decay down to 3e-5
Batch	64 sequences of 128 tokens (8,192 tokens per step)
Evaluation	Every 500 steps, averaged over 50 batches
Seed	1337
Hardware	Laptop CPU (AMD Ryzen 5 7535HS), no GPU

Overfit gate. Before long training, we trained on a tiny repeated batch; the loss fell from about 4.26 to below 0.01, showing gradients and target shifting work. [Re-run python -m src.train --overfit and confirm your numbers.] The starting loss of about 4.26 matches ln(67) = 4.20, the loss of a uniform guess over 67 symbols.

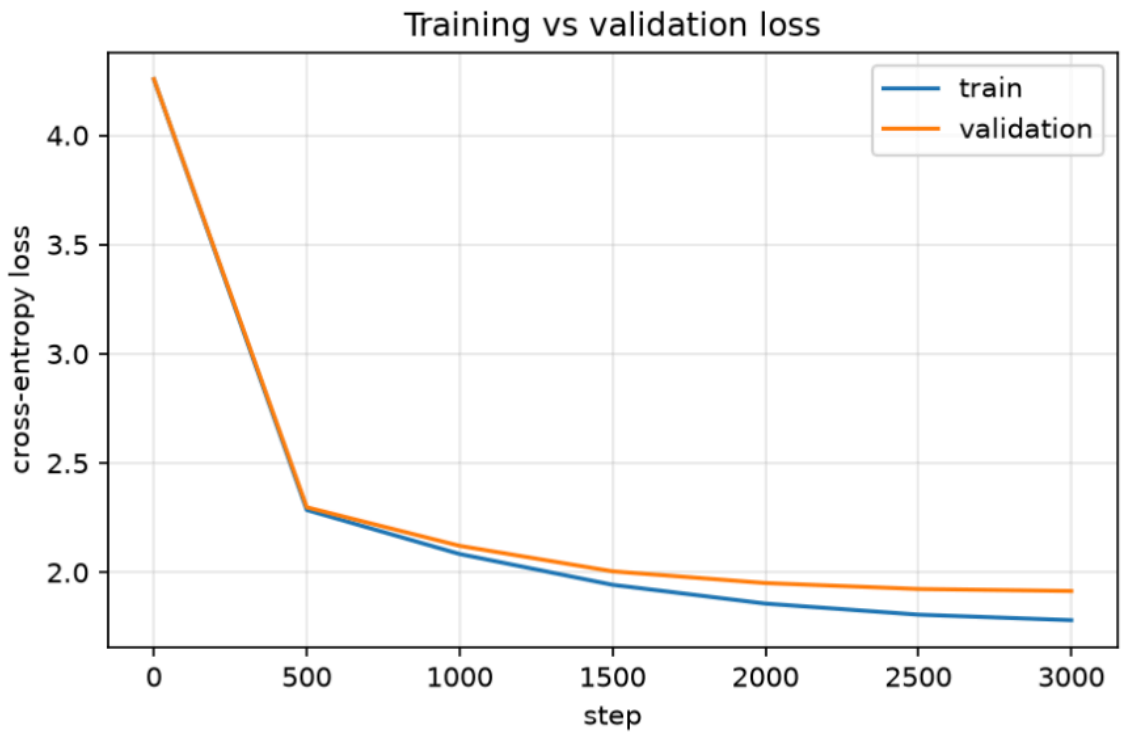
We chose a small configuration deliberately: the corpus is about 1 M characters, so a model of about 0.8 M parameters keeps the parameter-to-data ratio sensible, trains on a laptop CPU, and leaves room for the ablations.

6. Results

6.1 Training and validation loss

Run	Steps	Final train loss	Final val loss	Val perplexity	Wall time
Debug	500	2.3679	2.3712	10.71	844.7 s (4,849 tokens/s)
Main	3,000	1.7810	1.9144	6.78	4,539 s (about 75.7 min, 5,414 tokens/s)

After 3,000 steps, the validation loss fell to 1.9144 (perplexity 6.78), compared with 2.3712 (perplexity 10.71) after 500 steps and about 4.26 at initialisation. Training loss (1.7810) is 0.133 below validation loss, a small generalisation gap. The best validation checkpoint coincides with the final one.



The validation curve was still decreasing at the last evaluation, which suggests that longer training would improve the model further.

6.2 Generation samples

After 500 steps, the model has learned layout (speaker names, colons, line breaks) and some short words, but most words are garbled, e.g. 'Hijouces cair do curo dos clte cre that nee sth maked mas'. After 3,000 steps, from fixed prompts (temperature 0.8):

- **Prompt 'KING HENRY:\nWhat'** produced: 'What she a disme sout / Do be is his now sack Gremed, shave to be Rome, / What not his take if or and the and for the death'.
- **Prompt 'First Citizen:\nBefore we proceed'** produced: 'be lut's word of sir dence, / Then hat prooun. Whe the long and the prome thing: / In me the love in of the bother your so ach'.
- **Prompt 'The future of'** (not Shakespearean) produced: 'Rise the mole in theent. ANGELO: Come and since you wish have the read-right loset, And the king us that scoirs all I will and my know'.To be, or not to be, Hord to your band the rapties to past the folt–, For ROMUTES: It Godod, come, I grove your uppalins all your me, Theencion thees do ming the heave repon for the ase..SAUMLA: Thy how make of frie host sir, and the me he muth shall.,JUCETET: He th

Observation: speaker labels, line structure, punctuation, and many common words are correct, but many words are misspelled, and sentences carry no meaning.

Interpretation: a 0.8 M-parameter character-level model has learned local character statistics and formatting, but not semantics

.

6.3 Decoding comparison

On one prompt, we compared decoding strategies using distinct-3 (fraction of unique character trigrams) and the repeated-trigram rate.

Strategy	distinct-3	Repeated-trigram rate
Greedy	0.104	0.896
T = 0.5	0.582	0.418
T = 0.8	0.730	0.270
T = 1.2	0.871	0.129
T = 0.8, top-k 20	0.703	0.297

Observation: greedy decoding repeats heavily, and diversity rises monotonically with temperature.

Interpretation: greedy always picks the single most probable character and can fall into loops, while higher temperature flattens the distribution; very high temperature trades repetition for noisier text. Validation perplexity of the evaluated checkpoint: Fixed-prompt samples (temperature 0.8)

6.4 Attention inspection

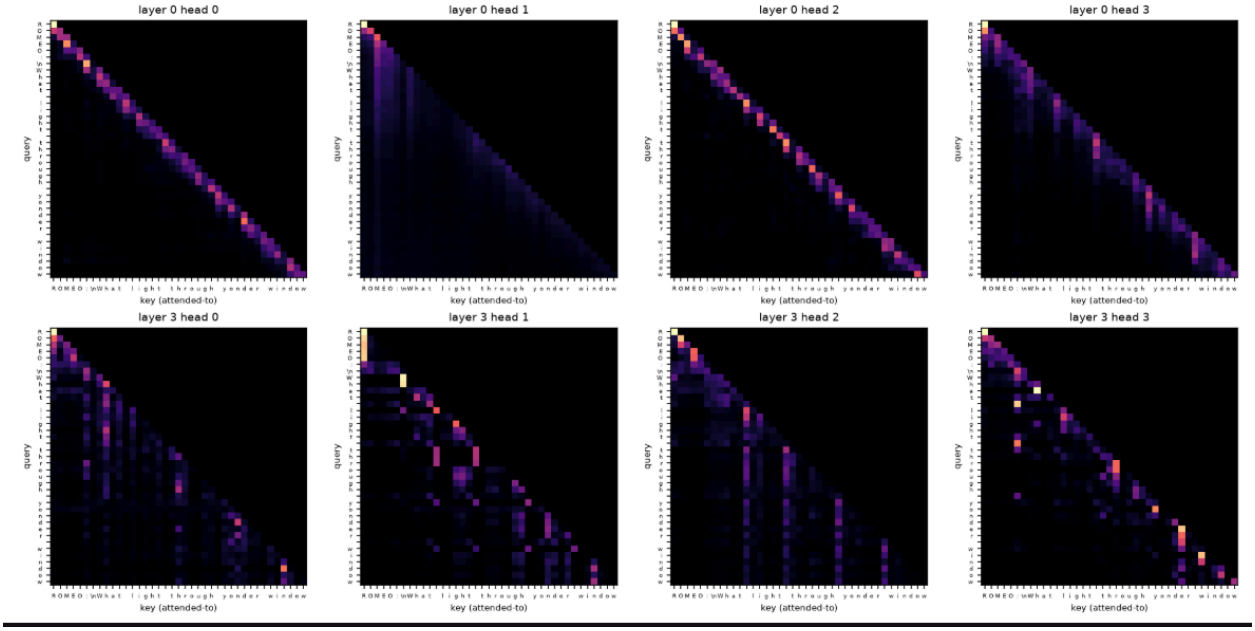


Figure 7: outputs/figures/fig7_attention.png.

The heatmap is zero above the diagonal, as required by the causal mask.

6.5 Sequential vs parallel depth: measurement

We timed a forward pass of a small RNN and an attention layer for increasing sequence length T.

T	RNN fwd (s)	Attn fwd (s)	attn matrix MB
32	0.0051	0.0014	0.12
64	0.0121	0.0024	0.50
128	0.0177	0.0039	2.00
256	0.0377	0.0150	8.00
512	0.1358	0.0841	32.00

Expect the attention matrix memory to grow quadratically (0.12 MB at T=32 to 32 MB at T=512 per head in our check). On a CPU at small sizes, the attention layer is not always faster in wall-clock time. In a quick CPU check, the RNN was faster at T=512. The O(1) sequential depth advantage matters most on parallel hardware (GPUs), while the $O(T^2)$ memory cost shows up at long sequences. This is an observation about our CPU test and not a general claim about GPUs.

6.6 MLM-style vs causal attention

We trained two identical small models (2 layers, d_model 64, context length 64) on next-token prediction, one with the causal mask and one with a bidirectional (leaky) mask. After 600 steps, the causal model reached a validation loss of 2.2448 (training loss 2.2313), while the bidirectional model reached 0.2180 (training loss 0.2083).

```
lm
Bidirectional (BERT) mask - every position sees every position:
 tensor([[1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1, 1, 1, 1]], dtype=torch.int32)
Causal (GPT) mask - position i sees j <= i:
 tensor([[1, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 0],
        [1, 1, 1, 1, 1, 1, 1, 1]], dtype=torch.int32)
causal_gpt          final train loss 2.2313 | val loss 2.2448
bidirectional_leaky  final train loss 0.2083 | val loss 0.2180

Observation: the bidirectional model reaches a much lower loss on next-token prediction.
Interpretation: it is not a better language model - it reads the target token through attention. That is why BERT uses
a *masked-token* objective and GPT needs the causal mask.
saved outputs/figures/mlm_vs_clm.png
```

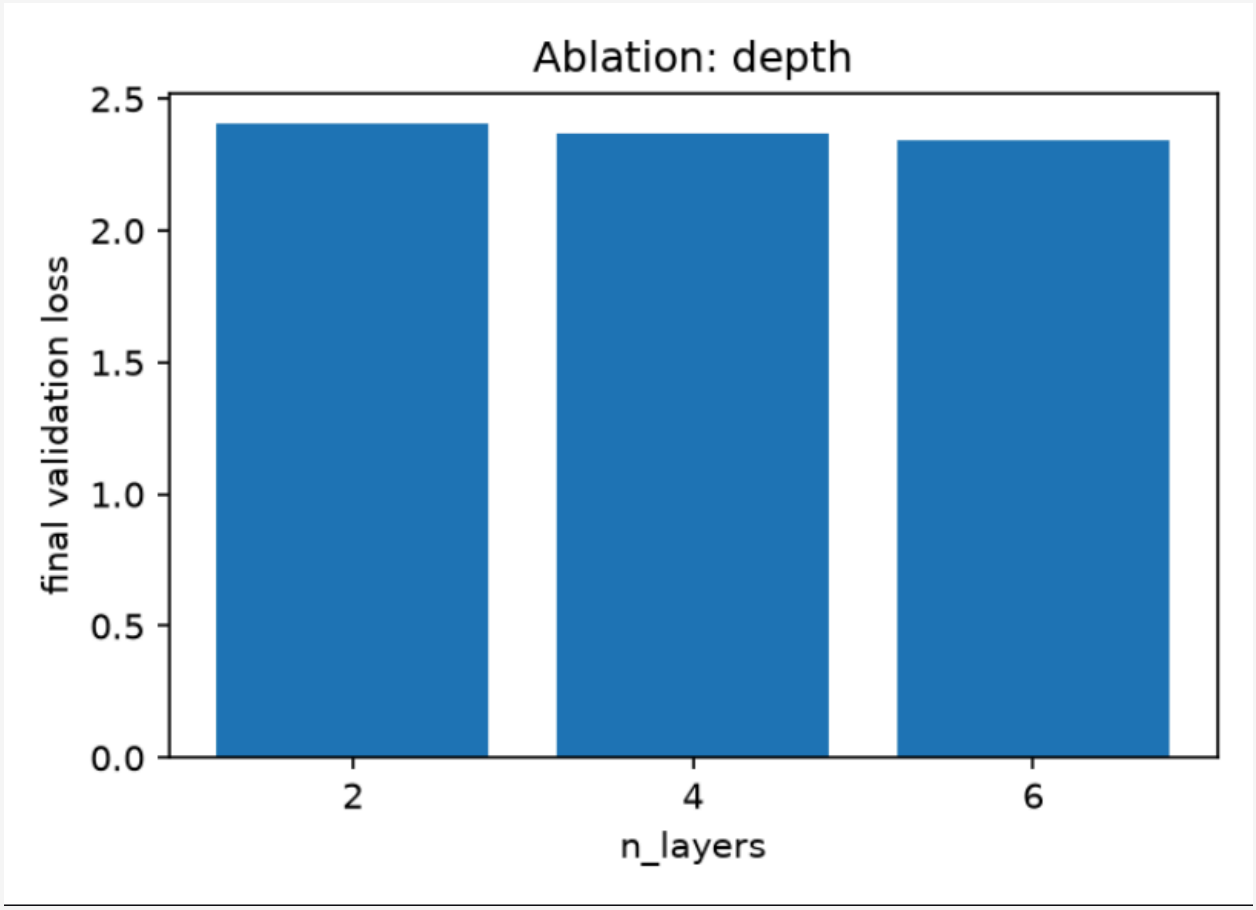
Observation: the bidirectional model's loss is about ten times lower than the causal model's.

Interpretation: this is not better language modelling. With a bidirectional mask, position t can attend to position t+1, which is exactly the target it must predict, so the model learns to copy the answer through attention instead of predicting it. A loss of 0.22 on character-level Shakespeare is implausibly low for a model this small, which supports this explanation. This is why GPT needs the causal mask and why BERT uses a masked-token objective, which hides the target tokens, rather than next-token targets.

7. Ablation Study

Each experiment changes one factor and keeps the rest fixed. All ablation runs use **500 steps** (a CPU constraint) and one random seed, so small differences should be read with caution.

7.1 Depth (number of blocks)

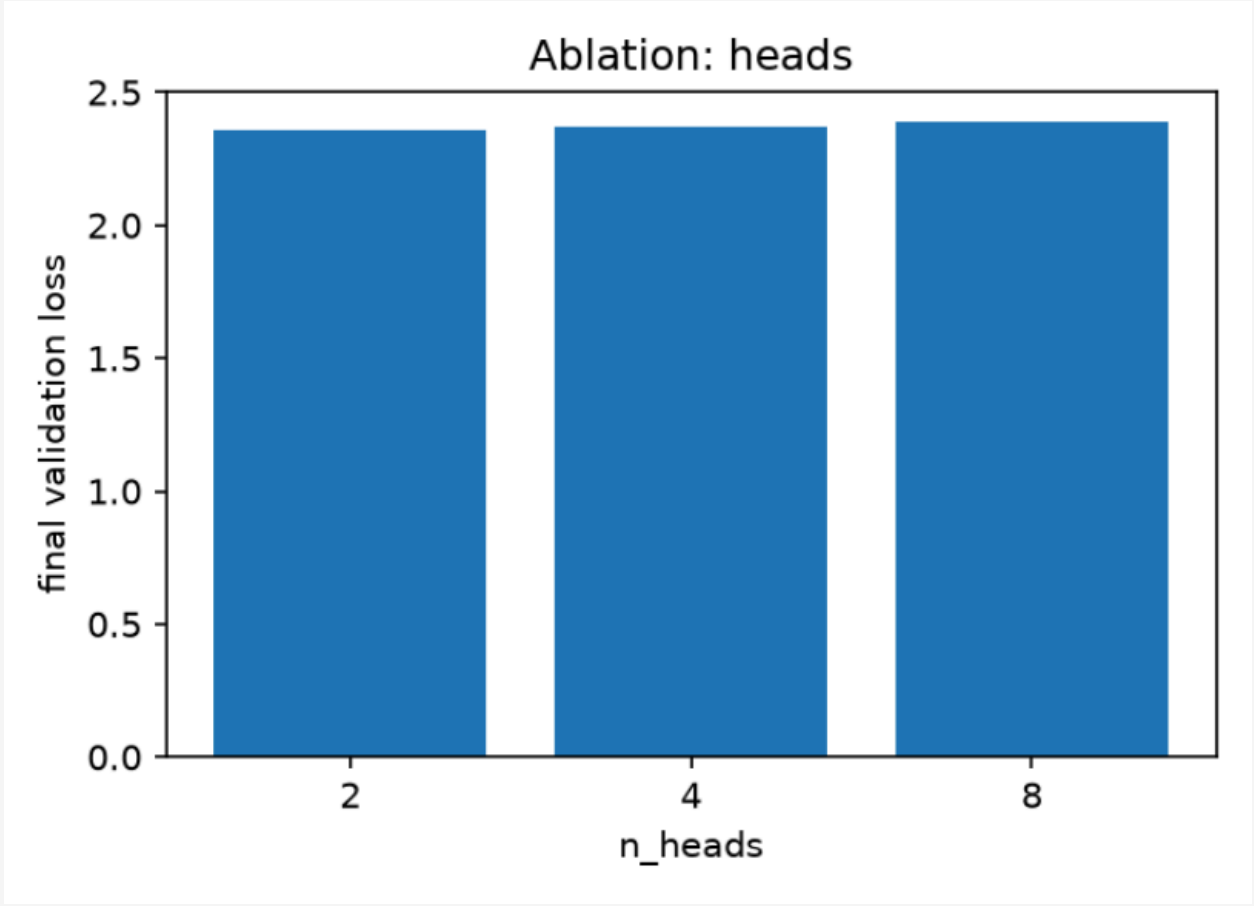


n_layers	Parameters	Train loss	Val loss	Val ppl	Time (s)	Tokens/s
2	413,952	2.3989	2.4034	11.061	443.1	9,244
4	810,496	2.3694	2.3689	10.686	932.9	4,391
6	1,207,040	2.3378	2.3404	10.385	1,299.9	3,151

Observation: going from 2 to 6 layers lowered validation loss from 2.4034 to 2.3404 (perplexity 11.06 to 10.39), while parameters tripled and training time rose about 2.9x (443 s to 1,300 s). The gain was 0.035 going from 2 to 4 layers and 0.029 going from 4 to 6.

Interpretation: extra layers add representational capacity, so loss improves, but each added layer helps a little less and costs a constant amount of compute; at 500 steps the larger models are also likely under-trained. With one seed, the differences between configurations are small, so we treat the ordering as suggestive rather than conclusive.

7.2 Number of heads



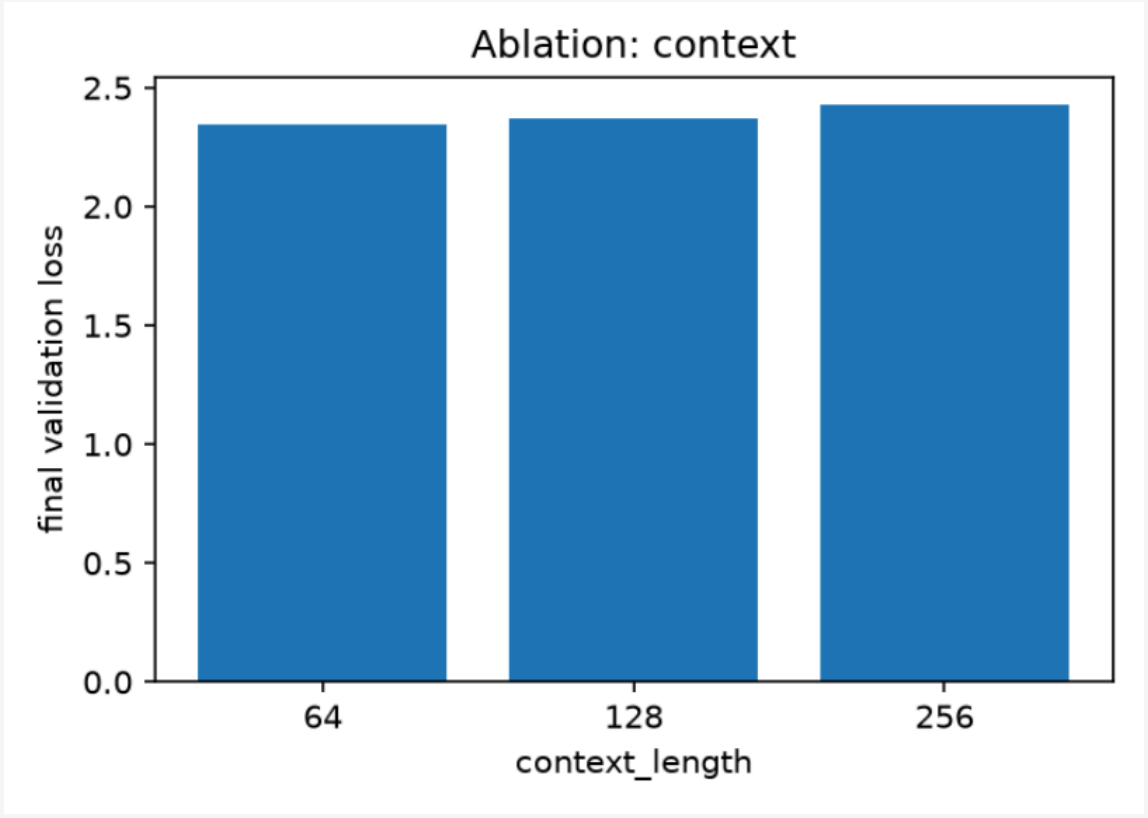
n_heads	d_head	Parameters	Train loss	Val loss	Val ppl	Time (s)	Tokens/s
2	64	810,496	2.3579	2.3596	10.587	682.4	6,002
4	32	810,496	2.3694	2.3689	10.686	828.3	4,945

8	16	810,496	2.3880	2.3864	10.874	1,175.5	3,484

Observation: The parameter count is identical (810,496) for all three settings, because d_model = 128 is only re-partitioned among heads. Validation loss increased slightly as the number of heads grew, from 2.3596 (2 heads) to 2.3689 (4 heads) and 2.3864 (8 heads), a total difference of 0.027 (perplexity 10.59 to 10.87). Training time rose from 682 s to 1,176 s (about 1.7x) and throughput fell from 6,002 to 3,484 tokens/s. The 4-head run reproduced the losses of the 4-layer depth experiment exactly, which confirms that runs are deterministic under a fixed seed.

Interpretation: With a fixed d_model, more heads means each head has fewer dimensions (64, 32, then 16), so each head can represent less on its own. At this small scale and after only 500 steps, fewer, wider heads may therefore be slightly easier to train. The slowdown is consistent with the attention weights having shape (B, H, T, T), so the score tensor grows with H, and with more, smaller matrix multiplications running on a CPU. We do not claim that 2 heads is better in general: the loss gap (0.027) is small, we used a single seed and 500 steps, and wall-clock times carry noise (the same 4-head configuration took 933 s in the depth experiment and 828 s here). We did not compare generation quality across head counts.

7.3 Context length



context_length	Tokens per step	Parameters	Train loss	Val loss	Val ppl	Time (s)	Tokens/s
64	4,096	810,496	2.3413	2.3470	10.454	321.4	6,372
128	8,192	810,496	2.3694	2.3689	10.686	828.3	4,945
256	16,384	810,496	2.4242	2.4252	11.305	2,430.8	3,370

T	Score tensor per layer

64	4.2 MB
128	16.8 MB
256	67.1 MB

Observation: The parameter count is identical (810,496) for all three settings, because context length does not change the weights (sinusoidal positions have no parameters). Validation loss increased with context length, from 2.3470 (64) to 2.3689 (128) and 2.4252 (256), i.e. perplexity 10.45, 10.69, and 11.31. Training time grew from 321 s to 828 s to 2,431 s. Throughput fell from 6,372 to 4,945 to 3,370 tokens/s, even though each step processes more tokens at longer context (the batch size of 64 sequences was fixed, so a step covers 4,096, 8,192 and 16,384 tokens). Measured peak memory is unavailable on CPU; the theoretical attention-score tensor grows fourfold each time the context doubles (4.2, 16.8 and 67.1 MB per layer).

Interpretation: The throughput drop is consistent with the $O(T^2)$ cost of attention: doubling T quadruples the score matrix, so each token becomes more expensive to process. Longer context did not help here. Possible reasons are that, with a fixed budget of 500 optimisation steps, the model has not yet learned to use long-range information, and that the same learning rate and schedule may suit the longer sequences less well. Because the loss is averaged over positions within each window, this comparison is not a controlled measure of how well long-range context is used. We did not test these explanations, and with a single seed and 500 steps the differences (0.02 to 0.06 in loss) are small, so we do not claim that shorter context is better in general.

8. Failure Analysis

1. **Misspelled words and meaningless sentences.** Even after 3,000 steps the model produces non-words ('mearst', 'scoirs'). Likely causes: character-level tokenisation forces the model to learn spelling from scratch, the model is tiny (0.81 M parameters), and the corpus is only about 1 M characters.
2. **Domain lock-in.** A non-Shakespearean prompt ('The future of') still yields archaic, play-like text with speaker labels. The model only saw one narrow domain, so it cannot move outside that distribution.
3. **Repetition under greedy decoding.** A repeated-trigram rate of 0.896 shows loop-like output. Greedy decoding keeps picking the most probable character, which reinforces repeated patterns.
4. **Limited statistical power in the ablations.** Single seed and only 500 steps, so small loss differences may be noise. A fuller study would average several seeds and train longer.

The generated text is an experimental artefact and not a factual claim.

9. Engineering Packaging

Reproducibility. From a clean checkout: pip install -r requirements.txt, then pytest -q, then python -m src.train --config configs/tiny_gpt.yaml, then python -m src. generate --prompt 'ROMEO:' --max-new-tokens 200. All hyperparameters live in configs/tiny_gpt.yaml and can be overridden on the command line (for example, max_steps=3000).

CLI. python -m src. generate prints the prompt, generated text, parameter count, context length, and temperature, as the handout requires.

```
Prompt: ROMEO:
Generated: ROMEO:
O my grooce to my sall spoy be thate thear proince,
But of my the our hone us heave thou not Bothert.

DUKE VINCENCE:
Net horle to should the behir demst it her:
And the alty maltale: not rething the
model parameters: 0.81M
context length: 128
temperature: 0.8
```

REST API. src/api.py serves FastAPI with GET /health, GET /info, and POST /generate, whose JSON body takes prompt, max_new_tokens, temperature, top_k, and top_p. At start-up, the service downloads the checkpoint from the URL in the CHECKPOINT_URL environment variable (the 3.27 MB file attached to GitHub release v1.0), so the repository stays small and contains no large artefacts or secrets.

Deployment. The service is containerised with the repository's Dockerfile and hosted on Render (free instance) at https://mini-gpt-yfoz.onrender.com. A live call to /generate with prompt 'ROMEO:', temperature 0.8, and top_k 40 returned HTTP 200 with generated text that began with a play-style speaker line.



/health reports model_loaded: true once the checkpoint is loaded. Free instances sleep after a period of inactivity, so the first request after idling can take about a minute.

10. Conclusion

A 0.81 M-parameter Transformer built from scratch and trained on a laptop CPU learned the formatting and local statistics of Shakespeare: validation perplexity fell from about 67 (a uniform guess) to about 10.7 after only 500 steps, and generated text reproduces speaker labels, line structure, and many real words. It did not learn meaning, which is expected at this scale. Our experiments support three points: (1) the causal mask is essential for next-token training, since removing it lets the model cheat; (2) adding depth lowered loss with diminishing returns at a steadily growing compute cost; and (3) decoding strategy has a large effect on repetition and diversity. The model was also packaged as a tested CLI and a deployed REST API, so the work is reproducible end to end. Future work: subword tokenisation, longer training, multiple seeds, and weight tying.

References

1. Vaswani et al., 'Attention Is All You Need', NeurIPS 2017.
2. Devlin et al., 'BERT: Pre-training of Deep Bidirectional Transformers', 2019.
3. Radford et al., 'Language Models are Unsupervised Multitask Learners' (GPT-2), 2019.
4. Ba, Kiros and Hinton, 'Layer Normalization', 2016.
5. Hendrycks and Gimpel, 'Gaussian Error Linear Units (GELUs)', 2016.
6. Loshchilov and Hutter, 'Decoupled Weight Decay Regularization' (AdamW), 2019.
7. Tiny Shakespeare dataset (Hugging Face dataset card; originating from Karpathy's char-rnn).